North South University

Department of Electrical & Computer Engineering

Project Report

Project Name: PerformTrack (Employee Performance Tracking System)

Github Repository Link: https://github.com/tjnblve-source/CSE327_Project-Employee_Performance_Tracking_System.git

Group Members:

Name	ID
Umme Habiba Haque	2211467042
Farhana Afrin Nahin	2221056042
Ilma Ibrahim	2321019042
Abu Bakar Siddik	2421953042

Submitted to:

Faculty Name: AKM Iqtidar Newaz

Faculty Initial: IqN

Course Code: CSE327.7

Course Title: Software Engineering

Introduction

PerformTrack is a business-focused platform that we designed with the aim to help organizations manage employee performance evaluation and promotion decisions efficiently. This online platform solves the problem of human errors, bias, and inefficiency caused by the manual handling of employee data and performance tracking by providing a system where employee data, performance records, attendance, and appraisal history can be stored and analysed.

Proposed Functional Requirements

Our initially suggested functional requirements were –

1. Ability of HR to register employees.
2. Storing of employees' personal and job information.
3. Ability of managers to record performance ratings.
4. Ability of managers to provide feedback and comments.
5. Tracking of employee attendance.
6. Maintenance of appraisal history.
7. Calculation of performance scores.
8. Identifying employees eligible for promotions.
9. Ability of Employees to view their performance reports.
10. Ability of HR to generate performance analytics reports.
11. Secure login for different users.

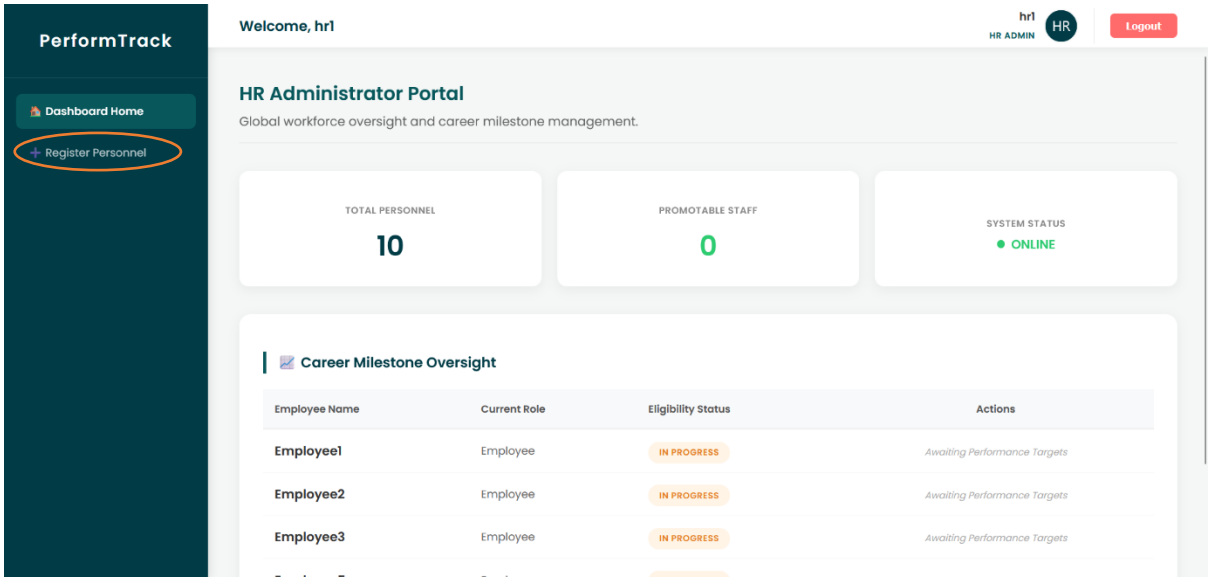
Completed Functional Requirements

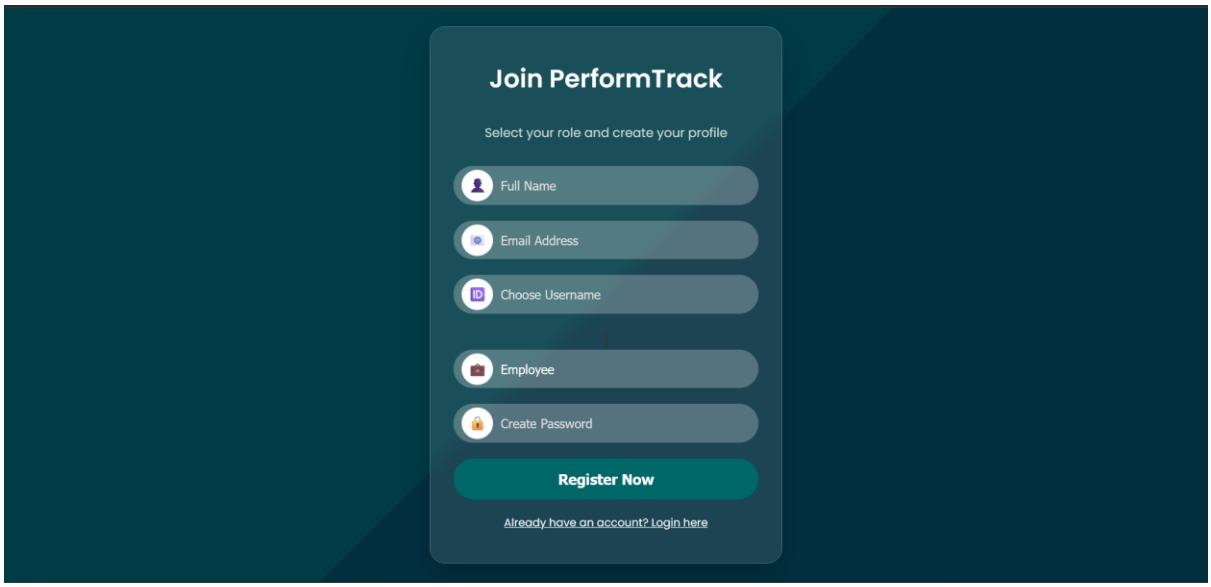
In our final project, we were able to implement some of the primary functional requirements with little modification to some of them.

1. <u>Ability of HR to register employees</u>

In our current system, while employees can register themselves into the database, HR can also register a new employee into the database.

```
<nav class="nav-container">
    <ul class="nav-list">
        <li onclick="location.href='index.php?page=dashboard'" class="nav-item <?php echo (!isset($_GET['page']) || $_GET['page'] == 'da
            🏠 Dashboard Home
        </li>

        <?php if($role === 'Manager'): ?>
            <li onclick="location.href='index.php?page=pending_appraisals'" class="nav-item">📝 Pending Appraisals</li>
            <li onclick="location.href='index.php?page=reports'" class="nav-item">📊 Performance Reports</li>
        <?php endif; ?>

        <?php if($role === 'HR Admin'): ?>
            <li onclick="location.href='index.php?page=register'" class="nav-item">➕ Register Personnel</li>
        <?php endif; ?>

        <?php if($role === 'Employee'): ?>
            <li onclick="location.href='index.php?page=process_achievement'" class="nav-item">🏆 My Achievements</li>
        <?php endif; ?>
    </ul>
</nav>
```





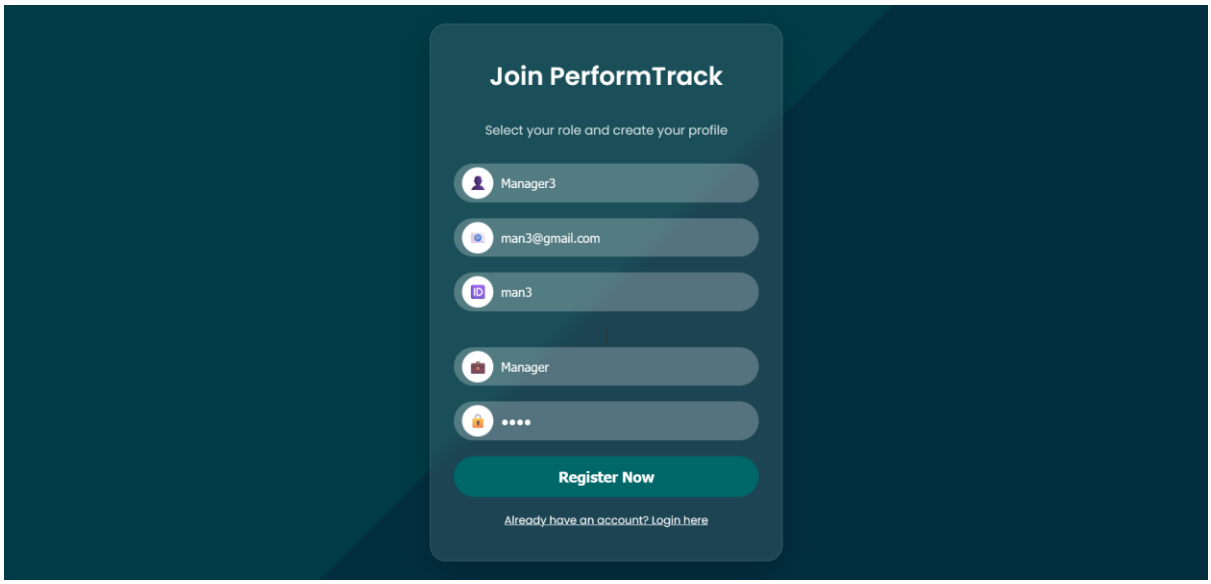
Our current system is very basic. So, the “Register Employee” option simply leads to the registration page.

2. Storing of employees’ information

Upon registration, all employees’ information are stored into the sql database.

```
case 'process_registration':
    $full_name = $_POST['full_name'];
    $email = $_POST['email'];
    $username = $_POST['username'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
    $role = $_POST['role'];

    $stmt = $conn->prepare("INSERT INTO user_account (UserName, Full_Name, Email, Password, Role) VALUES (?, ?, ?, ?, ?)");
    $stmt->bind_param("sssss", $username, $full_name, $email, $password, $role);

    if ($stmt->execute()) {
        $new_user_id = $conn->insert_id;
        if ($role === 'Employee') {
            $joining_date = date('Y-m-d');
            $emp_stmt = $conn->prepare("INSERT INTO employee (Employee_ID, Name, Email, Joining_Date, Promotion_Eligibility_Status, Attendance_
            $emp_stmt->bind_param("isss", $new_user_id, $full_name, $email, $joining_date);
            $emp_stmt->execute();
        }
        header("Location: index.php?page=login&registration=success");
        exit();
    }
    break;
```

This process in the index.php handles the insertion of registration data into the database.

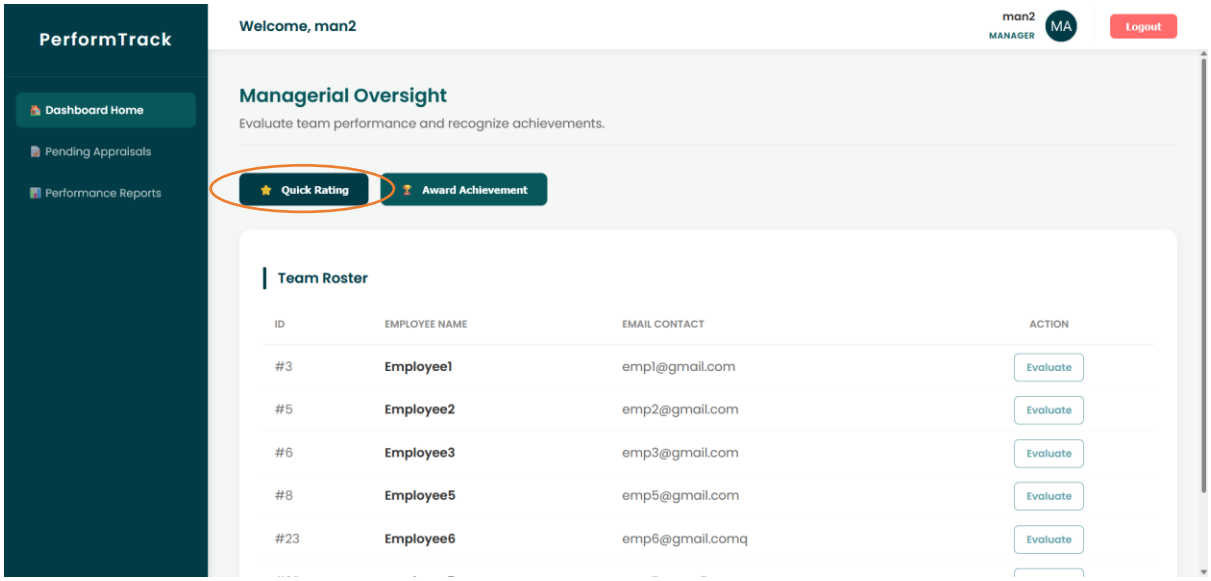


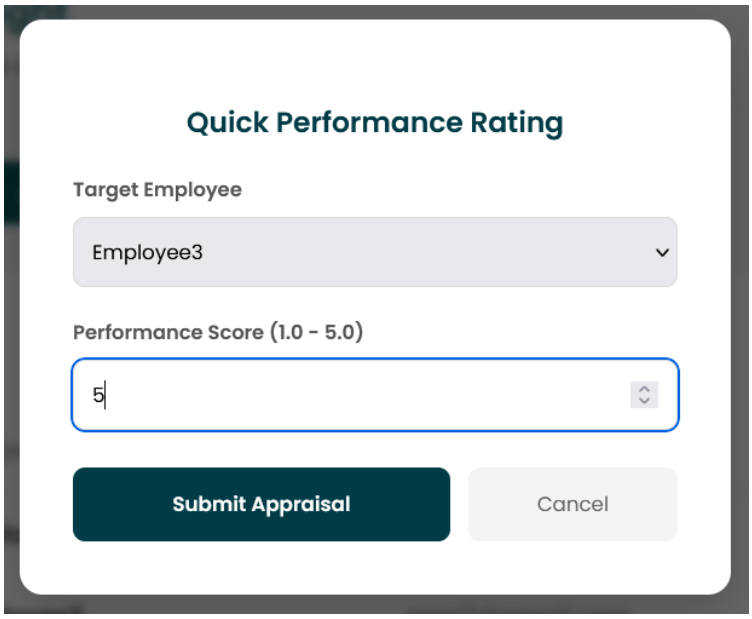
User_ID	UserName	Full_Name	Email	Phone	Password	Role
1	hr1	HR1	hr1@gmail.com	NULL	hr1	HR Admin
3	emp1	Employee1	emp1@gmail.com	NULL	emp1	Employee
4	man1	Manager1	man1@gmail.come	NULL	man1	Manager
5	emp2	Employee2	emp2@gmail.com	NULL	emp2	Employee
6	emp3	Employee3	emp3@gmail.com	NULL	emp3	Employee
7	emp4	Employee4	emp4@gmail.com	NULL	emp4	
8	emp5	Employee5	emp5@gmail.com	NULL	emp5	Employee
9	hr2	HR2	hr2@gmail.com	NULL	hr2	HR Admin
10	man2	Manager2	man2@gmail.com	NULL	man2	Manager
23	emp6	Employee6	emp6@gmail.comq	NULL	$2y$10$ABojoDJSo4bzlyluNUdV2OFpLXm1wrdL4pQTFcUkg7....	Employee
24	man3	Manager3	man3@gmail.com	NULL	$2y$10$92NURTqtk0kJhuOFI2bmwu2P5GYdV6TDITAtDEB4elg...	Manager

3. <u>Ability of managers to record performance ratings</u>

Managers are able to give quick performance rating to the employees and the rating is recorded in appraisal table as final score.

```
<div id="ratingModal" style="display:none; position: fixed; z-index: 9999; left: 0; top: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.6); backdrop-filter: blur(3px);">
    <div style="background: white; margin: 8% auto; padding: 35px; width: 400px; border-radius: 15px; box-shadow: 0 10px 30px rgba(0,0,0,0.2);">
        <h3 style="color: #003b46; margin-bottom: 20px; text-align: center;">Quick Performance Rating</h3>
        <form action="index.php?page=process_appraisal" method="POST">
            <label style="display:block; margin-bottom: 8px; font-size: 13px; font-weight: 600; color: #555;">Target Employee</label>
            <select name="emp_id" required style="width: 100%; padding: 12px; margin-bottom: 20px; border: 1px solid #ddd; border-radius: 8px; font-family: inherit;">
                <option value="" disabled selected>Choose from your team...</option>
                <?php
                $emp_list = $conn->query("SELECT User_ID, Full_Name FROM user_account WHERE Role = 'Employee'");
                while($e = $emp_list->fetch_assoc()) {
                    echo "<option value='{$e['User_ID']}'>".htmlspecialchars($e['Full_Name']).."</option>";
                }
                ?>
            </select>

            <label style="display:block; margin-bottom: 8px; font-size: 13px; font-weight: 600; color: #555;">Performance Score (1.0 - 5.0)</label>
            <input type="number" name="score" step="0.1" min="1" max="5" required placeholder="e.g. 4.5"
                   style="width: 100%; padding: 12px; margin-bottom: 25px; border: 1px solid #ddd; border-radius: 8px; box-sizing: border-box; font-family: inherit;">

            <div style="display: flex; gap: 12px;">
                <button type="submit" style="flex: 2; background: #003b46; color: white; border: none; padding: 14px; border-radius: 8px; cursor: pointer; font-weight: bold; font-fam
                <button type="button" onclick="document.getElementById('ratingModal').style.display='none'"
                        style="flex: 1; background: #f4f4f4; border: none; padding: 14px; border-radius: 8px; cursor: pointer; color: #666; font-family: inherit;">Cancel</button>
            </div>
        </form>
    </div>
</div>
```



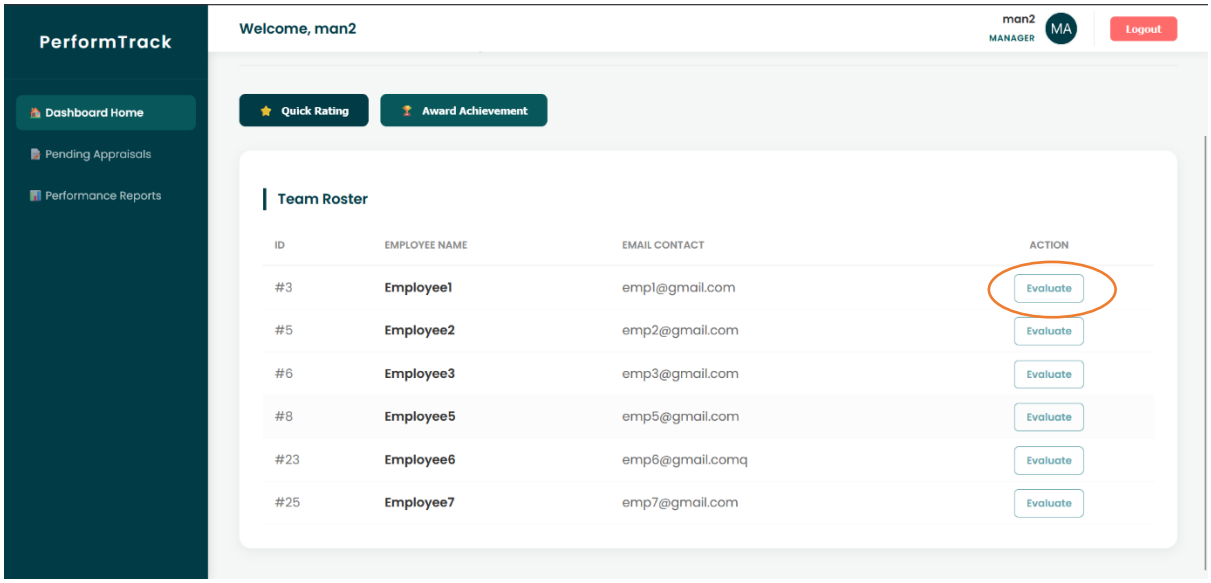


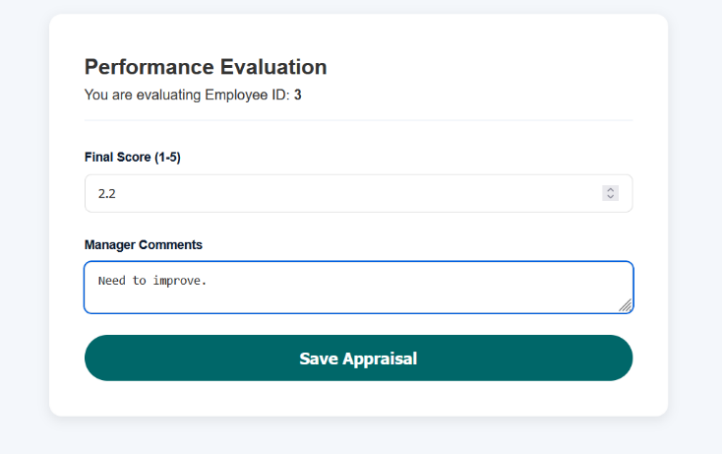
Appraisal_ID	Employee_ID	Final_Score	Review_Period
4	5	1.30	May 2026
5	6	5.00	May 2026
6	3	2.00	May 2026
7	3	2.00	May 2026
8	5	3.00	May 2026
9	5	3.00	May 2026
10	3	2.00	May 2026
11	8	5.00	May 2026
12	5	4.00	May 2026
13	7	5.00	May 2026
14	7	5.00	May 2026
15	6	3.00	May 2026
16	3	4.00	May 2026
17	6	5.00	May 2026

4. <u>Ability of managers to provide feedback and comments</u>

Managers can provide feedback and comments along with rating. However, the comments do not save into the database.

```php
case 'process_appraisal':
    $emp_id = $_POST['emp_id'];
    $score = $_POST['score'];
    $period = date("F Y");

    $att_res = $conn->query("SELECT Attendance_Percentage FROM employee WHERE Employee_ID = $emp_id");
    $att_data = $att_res->fetch_assoc();
    $attendance = $att_data['Attendance_Percentage'] ?? 0;

    $java_command = "java -cp bin com.performtrack.Main evaluate $score $attendance 2>&1";
    $java_output = shell_exec($java_command);

    $cleaned_output = ($java_output !== null) ? trim($java_output) : "";
    $new_status = (is_numeric($cleaned_output)) ? $cleaned_output : 0;

    $stmt = $conn->prepare("INSERT INTO appraisal (Employee_ID, Final_Score, Review_Period) VALUES (?, ?, ?)");
    $stmt->bind_param("ids", $emp_id, $score, $period);

    if ($stmt->execute()) {
        $conn->query("UPDATE employee SET Promotion_Eligibility_Status = $new_status WHERE Employee_ID = $emp_id");
        header("Location: index.php?page=dashboard&status=evaluated");
        exit();
    }
    break;
```

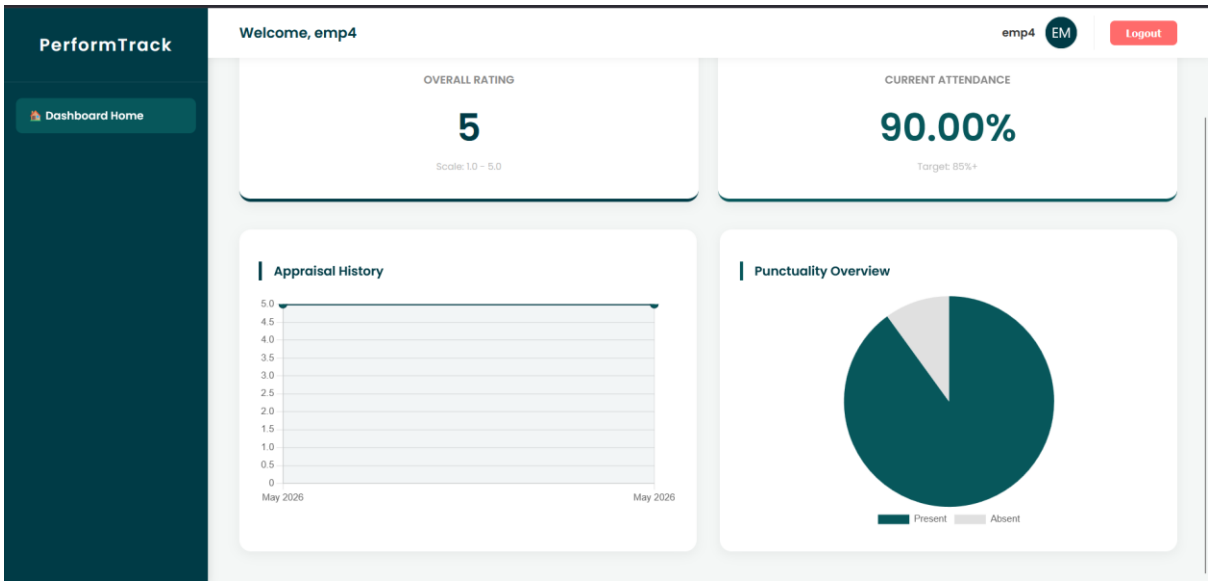




5. Charts of employee's monthly performance and attendance

In the employee dashboard, there is a line chart that shows the monthly employee performance and a pie chart to show employee attendance.

```
<script>
new Chart(document.getElementById('performanceLineChart').getContext('2d'), {
    type: 'line',
    data: {
        labels: <?php echo json_encode($months); ?>,
        datasets: [{
            label: 'Score',
            data: <?php echo json_encode($scores); ?>,
            borderColor: '#003b46',
            backgroundColor: 'rgba(0, 59, 70, 0.05)',
            fill: true,
            tension: 0.3,
            pointRadius: 5,
            pointBackgroundColor: '#07575b'
        }]
    },
    options: {
        responsive: true,
        plugins: { legend: { display: false } },
        scales: { y: { beginAtZero: true, max: 5 } }
    }
});

new Chart(document.getElementById('attendancePieChart').getContext('2d'), {
    type: 'pie',
    data: {
        labels: ['Present', 'Absent'],
        datasets: [{
            data: [<?php echo $attendance_val; ?>, <?php echo (100 - $attendance_val); ?>],
            backgroundColor: ['#07575b', '#e0e0e0'],
            borderWidth: 0
        }]
    },
    options: {
        responsive: true,
        plugins: { legend: { position: 'bottom' } }
    }
});
</script>
```



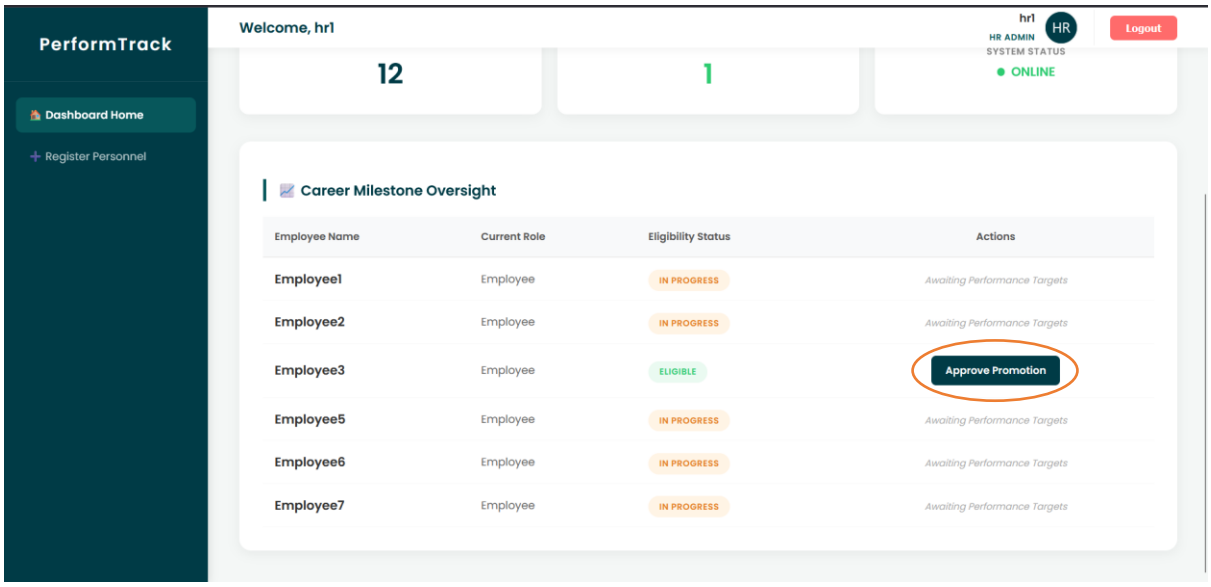
6. Identifying employees eligible for promotions upon HR approval

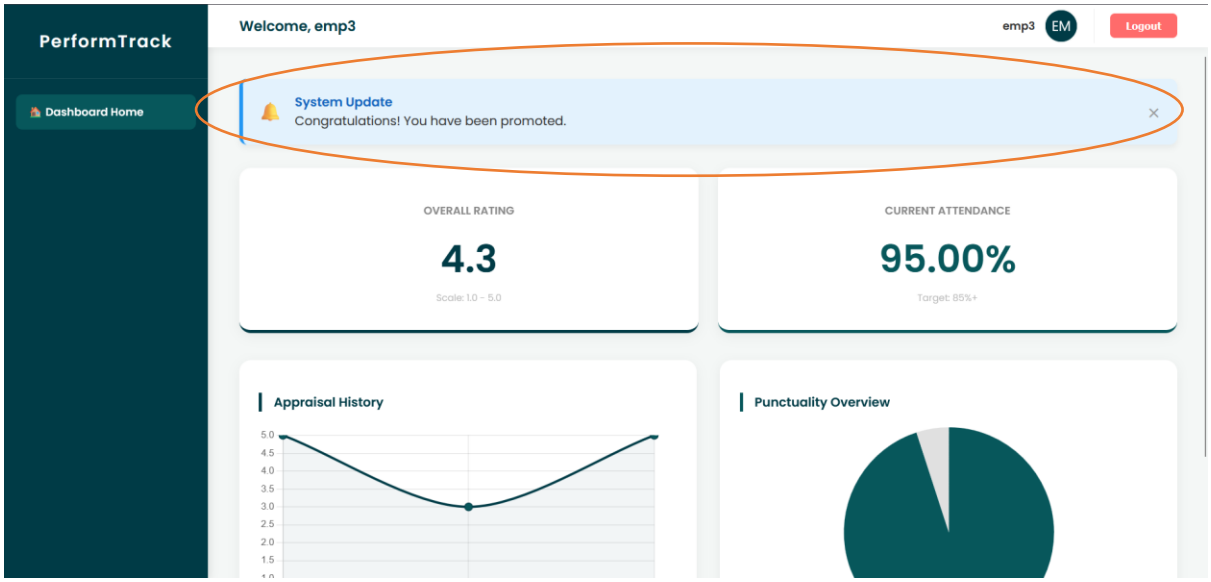
Given that an employee has their promotion eligibility status set to 1, the HR can approve the promotion and it will send the employee a notification of the approval when the employee logs into the system.

```
case 'approve_promotion':
    $id = $_GET['id'];
    $conn->query("UPDATE user_account SET Role = 'Senior Employee' WHERE User_ID = $id");
    $conn->query("UPDATE employee SET Promotion_Eligibility_Status = 0 WHERE Employee_ID = $id");
    shell_exec("java -cp bin com.performtrack.Main notifyPromotion $id");
    header("Location: index.php?page=dashboard&success=promoted");
    break;
```

backend > src > patterns > EmployeeNotificationService.java > ...

```java
package patterns;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;

public class EmployeeNotificationService implements PromotionObserver {
    private int targetEmployeeID;

    public EmployeeNotificationService(int employeeID) {
        this.targetEmployeeID = employeeID;
    }

    @Override
    public void update(String message) {
        String sql = "INSERT INTO notifications (Employee_ID, Message) VALUES (?, ?)";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setInt(parameterIndex: 1, this.targetEmployeeID);
            pstmt.setString(parameterIndex: 2, message);
            pstmt.executeUpdate();
            System.out.println("Notification sent to Database for Employee #" + targetEmployeeID);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

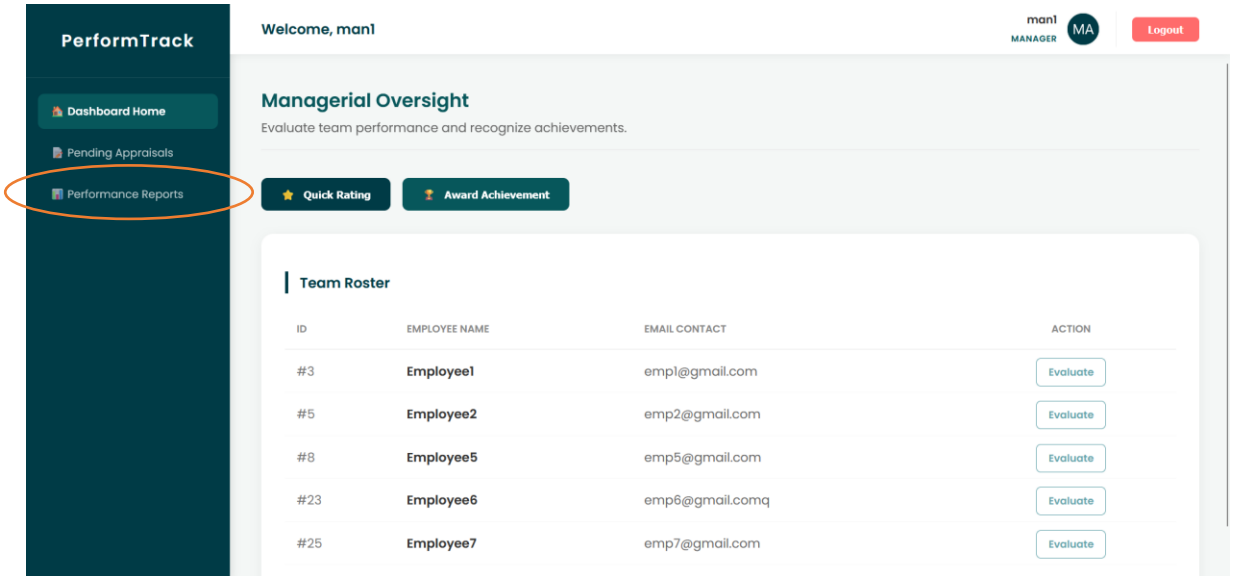


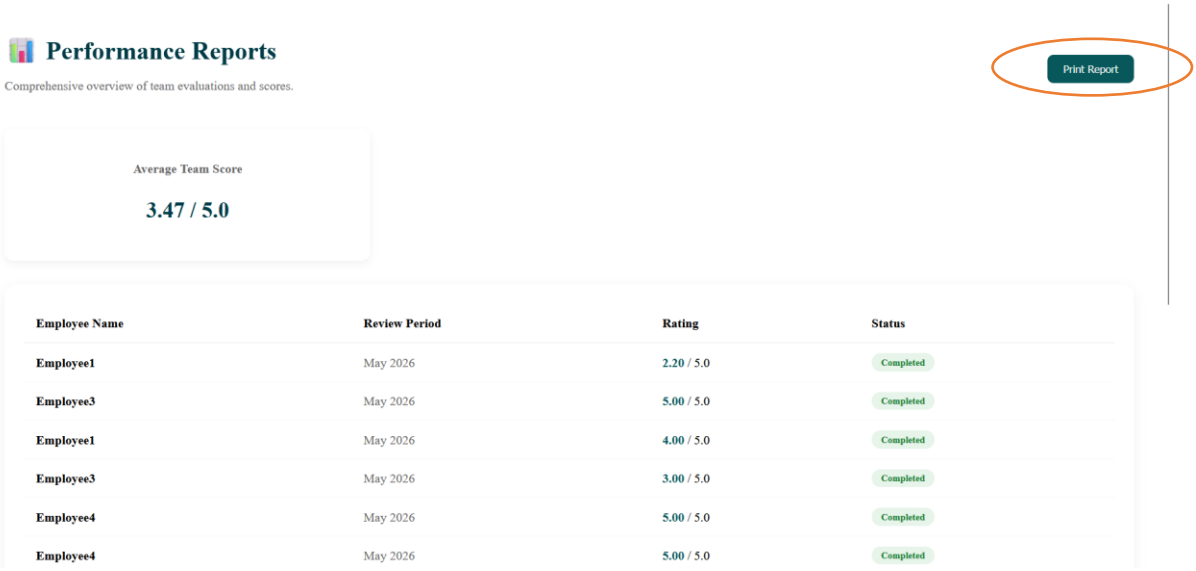


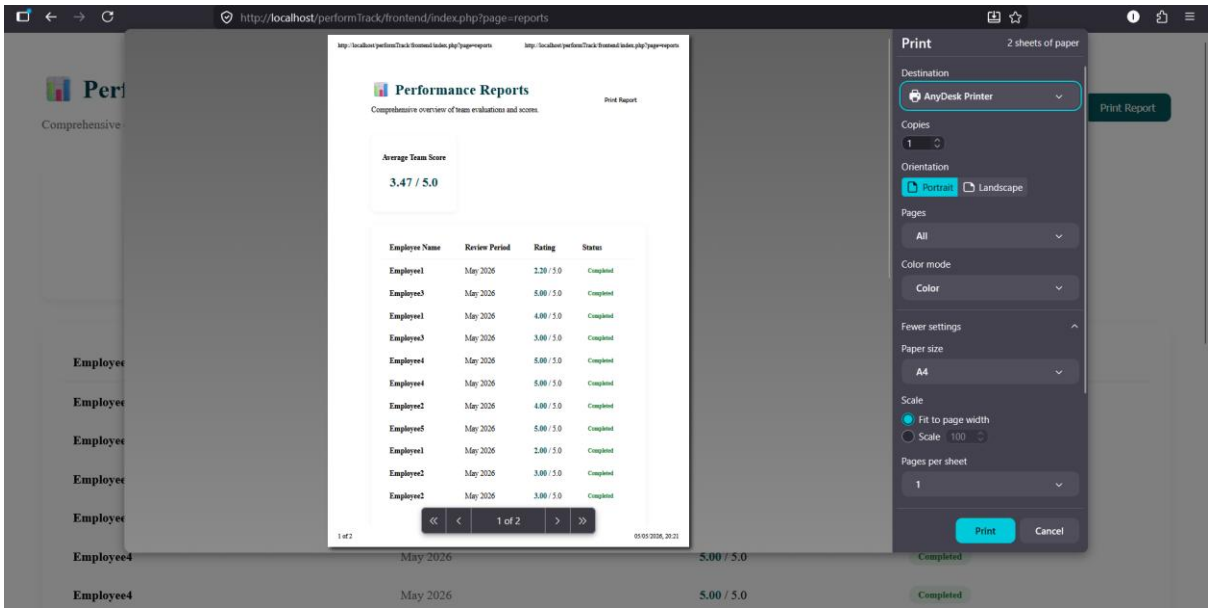
7. <u>Ability of Manager to generate performance analytics reports</u>

Manager can generate and print a performance analytics report of all employees. It is a basic list of the appraisal table.

```
case 'reports':
    $user_role = $_SESSION['role'] ?? 'Manager';
    $report_data = shell_exec("java -cp bin com.performtrack.Main generateReport $user_role 2>&1");
    include 'views/reports.php';
    break;
```

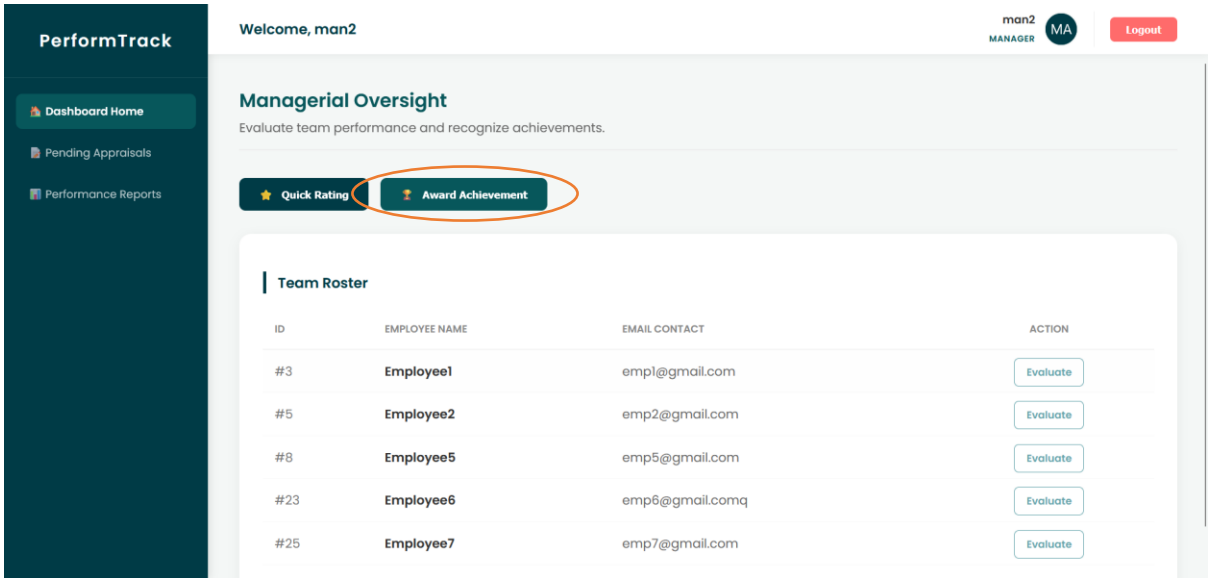


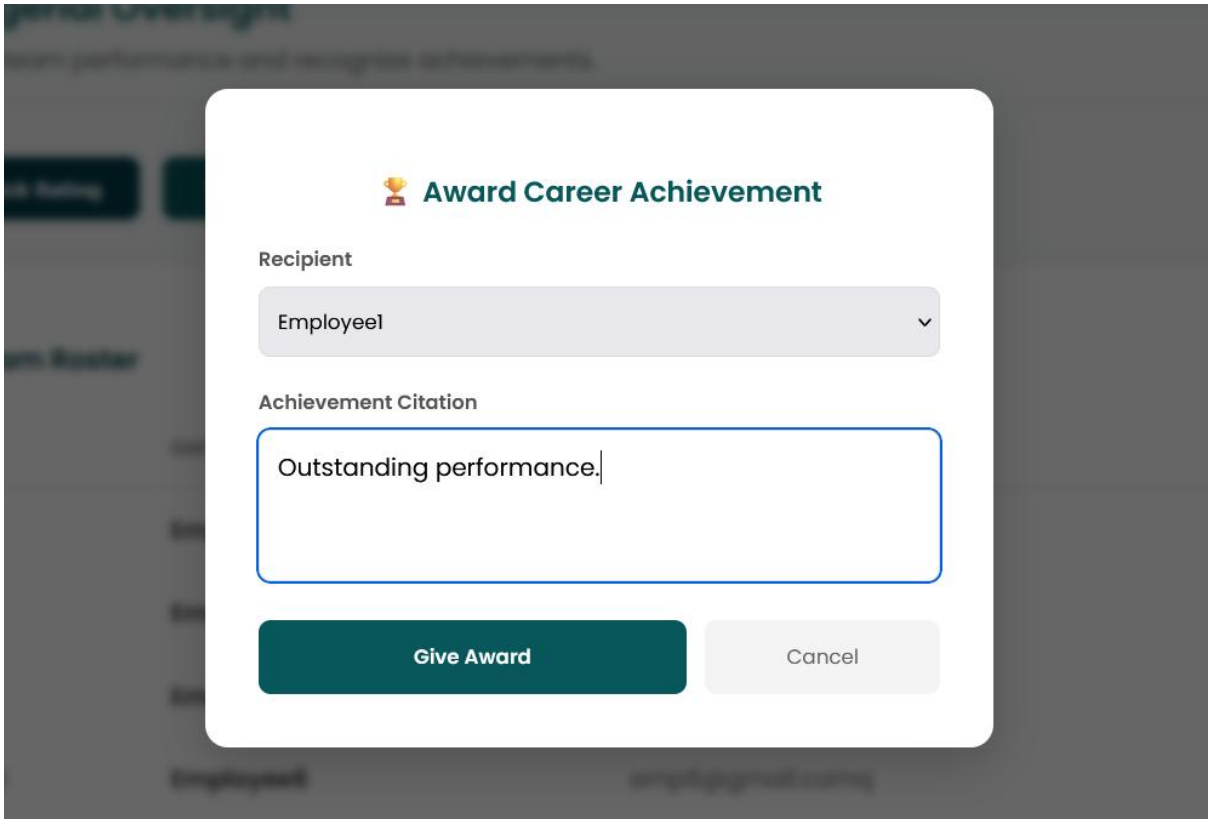


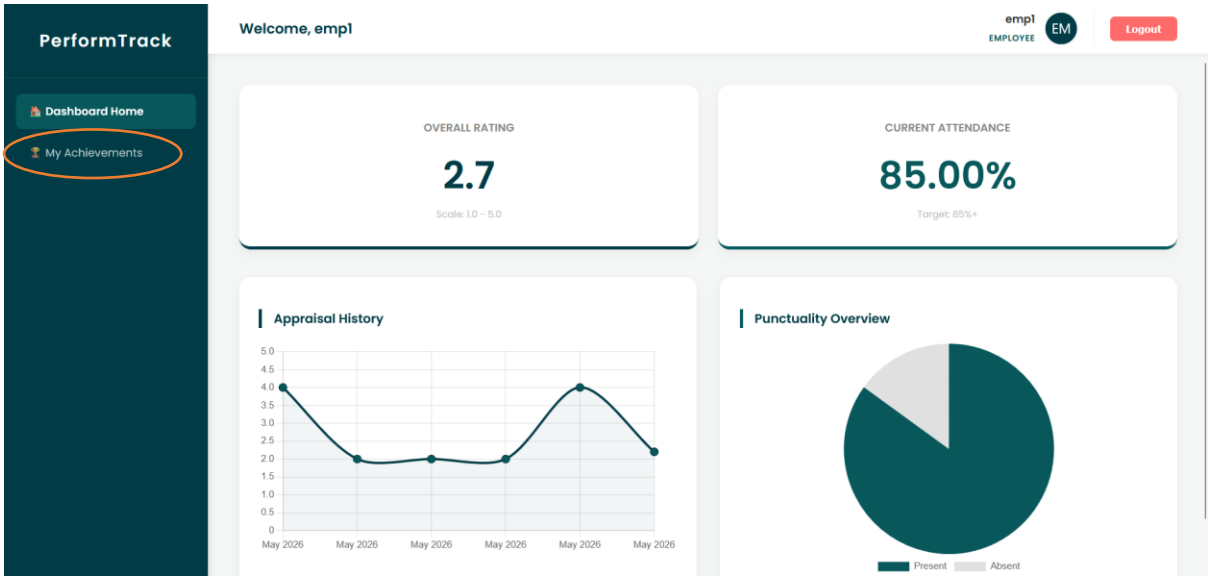


8. <u>Ability of Manager to give Employees achievements</u>

Managers can give employees achievements that save into the achievements table in the database.


PerformTrack
Welcome, man2
man2
MANAGER
MA
Logout
Dashboard Home
Pending Appraisals
Performance Reports
Managerial Oversight
Evaluate team performance and recognize achievements.
Quick Rating
Award Achievement
Team Roster
ID
EMPLOYEE NAME
EMAIL CONTACT
ACTION
#3
Employee1
emp1@gmail.com
Evaluate
#5
Employee2
emp2@gmail.com
Evaluate
#8
Employee5
emp5@gmail.com
Evaluate
#23
Employee6
emp6@gmail.comq
Evaluate
#25
Employee7
emp7@gmail.com
Evaluate


Award Career Achievement
Recipient
Employee1
Achievement Citation
Outstanding performance.
Give Award
Cancel



🏆 My Achievements

A record of your hard work and recognized milestones.

Recognition Milestone

f

Earned on: Date not recorded

Recognition Milestone

Outstanding performance.

Earned on: Date not recorded

Achievement_ID	Employee_ID	Achievement_Date	Description
2	6	NULL	Employee of the Month
3	5	NULL	Y
4	7	NULL	fhffh
5	3	NULL	f
7	3	NULL	Outstanding performance.

Proposed Non-Functional Requirements

Security

- Only authorized users can access the system.
- Employee information must be protected.

Performance

- The system should respond quickly to user requests.
- The system should handle multiple users simultaneously.

Usability

- The interface should be easy to use.
- Users should be able to navigate the system easily.

Reliability

- The system should store data without loss.
- The system should operate with minimal downtime.

Scalability

- The system should support increasing numbers of employees.

Maintainability

- The system should be easy to update and maintain.

Completed Non-Functional Requirements

Performance

- The system responds quickly to user requests.
- The system can handle multiple users simultaneously.

Usability

- The interface is easy to use.
- Users are able to navigate the system easily.

Reliability

- The system stores data that have connection path to the database without loss.
- The system operates with minimal downtime.

Implementation of Different Design Patterns

We tried to implement the following design patterns into our project –

- Singleton (Thread-safe) for User registration

backend > src > config > Database.java > Database

```java
package config;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Database {
    private static Database instance;
    private Connection connection;
    private String url = "jdbc:mysql://localhost:3306/performtrack";
    private String user = "root";
    private String password = "";

    private Database() throws SQLException {
        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
            this.connection = DriverManager.getConnection(url, user, password);
        } catch (ClassNotFoundException e) {
            throw new SQLException("JDBC Driver not found: " + e.getMessage());
        }
    }

    public static synchronized Database getInstance() throws SQLException {
        if (instance == null || instance.connection.isClosed()) {
            instance = new Database();
        }
        return instance;
    }

    public Connection getConnection() { return connection; }
}
```

- Factory for User object handling

backend > src > patterns > UserFactory.java > UserFactory

```java
package patterns;

import models.*;
import config.Database;
import java.sql.*;

public class UserFactory {
    public static UserAccount getUser(int userID) {
        String query = "SELECT UserName, Role FROM user_account WHERE User_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, userID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                String name = rs.getString(columnLabel: "UserName");
                String role = rs.getString(columnLabel: "Role");
                return switch (role) {
                    case "Manager" -> new Manager(userID, name);
                    case "HR Admin" -> new HumanResource(userID, name);
                    default -> new Employee(userID, name);
                };
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return null;
    }
}
```

- Observer for Employee Promotion Approval and Notification

backend > src > patterns > PromotionObserver.java > { } patterns

```java
package patterns;

public interface PromotionObserver {
    void update(String message);
}
```

backend > src > patterns > EmployeeNotificationService.java > EmployeeNotificationService

```java
package patterns;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;

public class EmployeeNotificationService implements PromotionObserver {
    private int targetEmployeeID;

    public EmployeeNotificationService(int employeeID) {
        this.targetEmployeeID = employeeID;
    }

    @Override
    public void update(String message) {
        String sql = "INSERT INTO notifications (Employee_ID, Message) VALUES (?, ?)";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setInt(parameterIndex: 1, this.targetEmployeeID);
            pstmt.setString(parameterIndex: 2, message);
            pstmt.executeUpdate();
            System.out.println("Notification sent to Database for Employee #" + targetEmployeeID);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- Decorator for Report

backend > src > patterns > ReportDecorator.java > HighPerformerDecorator

```java
package patterns;

import models.Report;

public abstract class ReportDecorator extends Report {
    protected Report decoratedReport;
    public ReportDecorator(Report report) { this.decoratedReport = report; }

    @Override
    public String getDataSummary() {
        return decoratedReport.getDataSummary();
    }
}

class HighPerformerDecorator extends ReportDecorator {
    public HighPerformerDecorator(Report report) { super(report); }
    @Override
    public String getDataSummary() {
        return super.getDataSummary() + " | [BADGE: STAR PERFORMER]";
    }
}
```

- Proxy for Report Generation and Access

```
backend > src > patterns > ReportProxy.java > ReportProxy
package patterns;

import models.Report;
import models.UserAccount;

public class ReportProxy extends Report {
    private Report realReport;
    private UserAccount currentUser;
    private int requestedReportID;

    /**
     * @param user The user attempting to access the report.
     * @param reportID The ID of the report requested from the DB.
     */
    public ReportProxy(UserAccount user, int reportID) {
        this.currentUser = user;
        this.requestedReportID = reportID;
    }

    @Override
    public String getDataSummary() {
        if (hasAccess()) {
            if (realReport == null) {
                System.out.println(x: "Proxy: Access granted. Loading high-resolution report data from MySQL...");
                realReport = new Report(requestedReportID);
            }
            return realReport.getDataSummary();
        } else {
            return "ACCESS DENIED: You do not have the 'HR Admin' role required to view this report.";
        }
    }

    @Override
    public void exportPDF() {
        if (hasAccess()) {
            if (realReport == null) {
                realReport = new Report(requestedReportID);
            }
            realReport.exportPDF();
        } else {
            System.out.println("Proxy Alert: Unauthorized PDF export attempt by User ID " + currentUser.getUserID());
        }
    }

    private boolean hasAccess() {
        return currentUser != null && "HR Admin".equalsIgnoreCase(currentUser.getRole());
    }
}
```

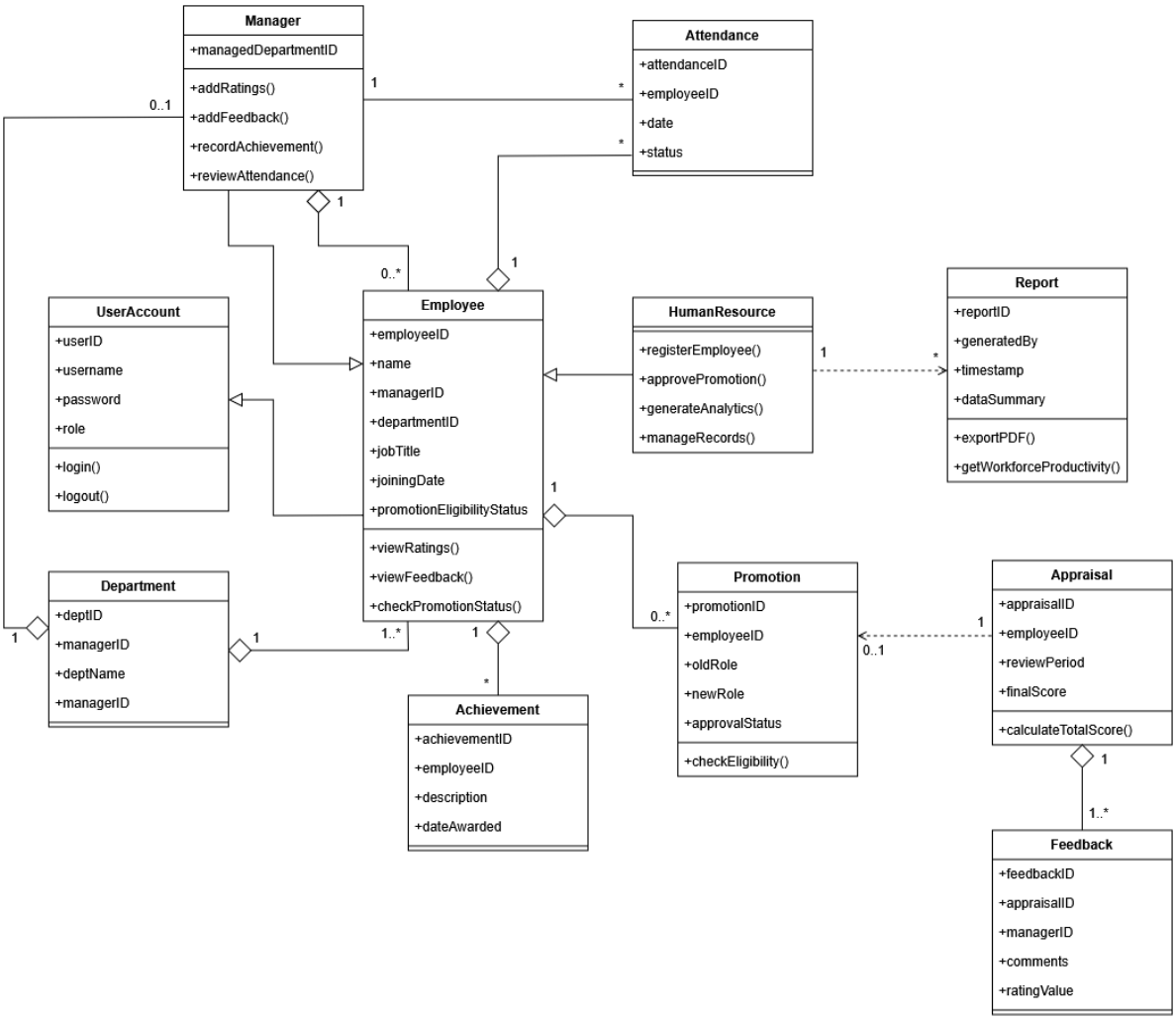
- Strategy for Performance Scoring

```
backend > src > patterns > ScoringStrategy.java > ScoringStrategy
package patterns;

public interface ScoringStrategy {
    /**
     * Calculates the score based on raw performance data.
     * @param rawScore The base score provided by the manager in the PHP form.
     * @param attendance The attendance percentage from the database.
     * @return The final processed score.
     */
    double calculate(double rawScore, double attendance);
}
```

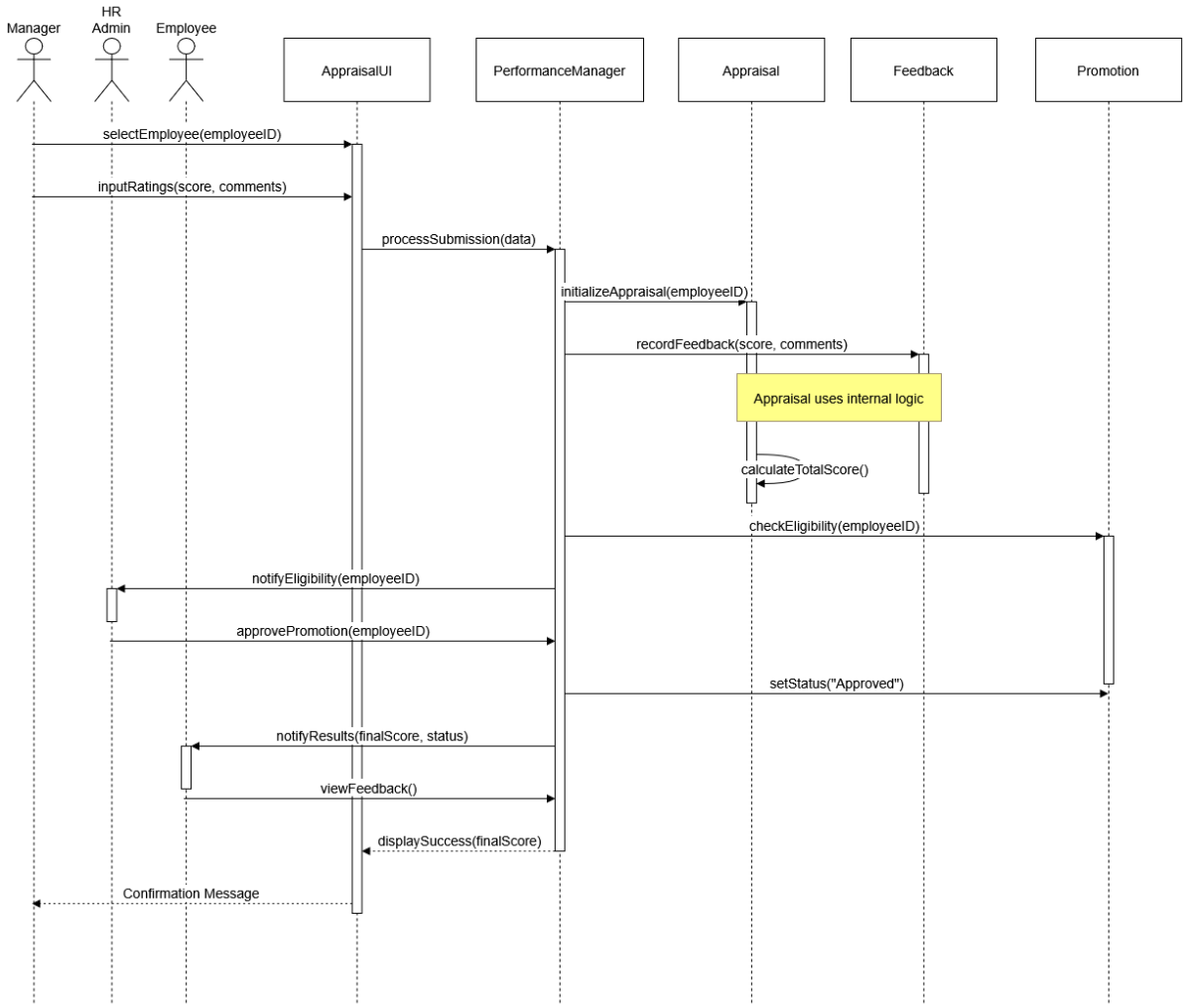
Due to time constraint and lack of practice, we were not able to implement some of these design patterns properly.

Class and Sequence Diagrams and Actual Implementation

Class Diagram:



Sequence Diagram:



Although we were able to create most of the classes and their respective methods according to the Class Diagram, the time constraint and our lack of understanding regarding the implementation of java methods on localhost server rendered many of these methods non-functional.

UserAccount Class

backend > src > models > UserAccount.java > UserAccount > login(String, String)

```java
package models;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UserAccount {
    protected int userID;
    protected String username;
    protected String password;
    protected String role;

    public UserAccount() {}

    public UserAccount(int userID, String username, String role) {
        this.userID = userID;
        this.username = username;
        this.role = role;
    }

    public boolean login(String inputUsername, String inputPassword) {
        String query = "SELECT * FROM user_account WHERE UserName = ? AND Password = ?";

        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);

            pstmt.setString(parameterIndex: 1, inputUsername);
            pstmt.setString(parameterIndex: 2, inputPassword);

            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                this.userID = rs.getInt(columnLabel: "User_ID");
                this.username = rs.getString(columnLabel: "UserName");
                this.role = rs.getString(columnLabel: "Role");
                System.out.println("Login Successful. Welcome, " + this.username + " (" + this.role + ")");
                return true;
            }
        } catch (SQLException e) {
            System.err.println("Database error during login: " + e.getMessage());
        }

        System.out.println(x: "Invalid credentials.");
        return false;
    }

    public void logout() {
        System.out.println("User " + this.username + " has logged out.");
        this.userID = 0;
        this.username = null;
        this.role = null;
    }

    public int getUserID() { return userID; }
    public String getUsername() { return username; }
    public String getRole() { return role; }

    public void setPassword(String password) { this.password = password; }
}
```

Employee Class

backend > src > models > Employee.java > Employee > viewRatings()

```java
package models;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class Employee extends UserAccount {
    protected int employeeID;
    protected String name;
    protected int managerID;
    protected int departmentID;
    protected String jobTitle;
    protected String joiningDate;
    protected boolean promotionEligibilityStatus;

    public Employee(int userID, String username) {
        super(userID, username, role: "Employee");
        this.employeeID = userID;
        loadEmployeeData();
    }

    private void loadEmployeeData() {
        String query = "SELECT * FROM employee WHERE Employee_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.employeeID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                this.name = rs.getString(columnLabel: "Name");
                this.departmentID = rs.getInt(columnLabel: "Department_ID");
                this.managerID = rs.getInt(columnLabel: "Manager_ID");
                this.jobTitle = rs.getString(columnLabel: "Job_Title");
                this.joiningDate = rs.getString(columnLabel: "Joining_Date");
                this.promotionEligibilityStatus = rs.getBoolean(columnLabel: "Promotion_Eligibility_Status");
            }
        } catch (SQLException e) {
            System.err.println("Error loading employee profile: " + e.getMessage());
        }
    }

    public void viewRatings() {
        String query = "SELECT Final_Score, Review_Period FROM appraisal WHERE Employee_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.employeeID);
            ResultSet rs = pstmt.executeQuery();

            System.out.println(x: "--- Your Performance Ratings ---");
            while (rs.next()) {
                System.out.println("Period: " + rs.getString(columnLabel: "Review_Period") +
                                   " | Score: " + rs.getDouble(columnLabel: "Final_Score"));
            }
        } catch (SQLException e) {
            System.err.println("Error fetching ratings: " + e.getMessage());
        }
    }

    public void viewFeedback() {
        String query = "SELECT f.Comments, f.Rating_Value FROM feedback f " +
                       "JOIN appraisal a ON f.Appraisal_ID = a.Appraisal_ID " +
                       "WHERE a.Employee_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.employeeID);
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                System.out.println("Rating: " + rs.getInt(columnLabel: "Rating_Value") + "/5");
                System.out.println("Manager Comment: " + rs.getString(columnLabel: "Comments"));
            }
        } catch (SQLException e) {
            System.err.println("Error fetching feedback: " + e.getMessage());
        }
    }

    public void checkPromotionStatus() {
        if (this.promotionEligibilityStatus) {
            System.out.println(x: "Status: You are currently ELIGIBLE for promotion.");
        } else {
            System.out.println(x: "Status: Not yet eligible for promotion.");
        }
    }

    public int getEmployeeID() { return employeeID; }
    public String getName() { return name; }
    public String getJobTitle() { return jobTitle; }
}
```

HumanResource Class

backend > src > models > HumanResource.java > HumanResource > generateAnalytics()

```java
package models;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class HumanResource extends Employee {

    public HumanResource(int userID, String username) {
        super(userID, username);
        this.role = "HR Admin";
    }

    public void registerEmployee(String name, String email, String jobTitle, int deptID) {
        String query = "INSERT INTO employee (Name, Email, Job_Title, Department_ID, Joining_Date) VALUES (?, ?, ?, ?, CURDATE())";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setString(parameterIndex: 1, name);
            pstmt.setString(parameterIndex: 2, email);
            pstmt.setString(parameterIndex: 3, jobTitle);
            pstmt.setInt(parameterIndex: 4, deptID);

            pstmt.executeUpdate();
            System.out.println("Employee " + name + " registered successfully.");
        } catch (SQLException e) {
            System.err.println("Error registering employee: " + e.getMessage());
        }
    }

    public void approvePromotion(int promotionID, String status) {
        String query = "UPDATE promotion SET Approval_Status = ? WHERE Promotion_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setString(parameterIndex: 1, status);
            pstmt.setInt(parameterIndex: 2, promotionID);

            int rows = pstmt.executeUpdate();
            if (rows > 0) {
                System.out.println("Promotion " + promotionID + " status updated to: " + status);
            }
        } catch (SQLException e) {
            System.err.println("Error updating promotion: " + e.getMessage());
        }
    }

    public void generateAnalytics() {
        String query = "SELECT AVG(Final_Score) as AvgScore FROM appraisal";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                System.out.println(x: "--- Workforce Analytics ---");
                System.out.println("Average Company Performance Score: " + rs.getDouble(columnLabel: "AvgScore"));
            }
        } catch (SQLException e) {
            System.err.println("Error generating analytics: " + e.getMessage());
        }
    }

    public void manageRecords(int targetEmployeeID) {
        String query = "SELECT * FROM employee WHERE Employee_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, targetEmployeeID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                System.out.println("Managing Record for: " + rs.getString(columnLabel: "Name"));
                System.out.println("Current Job Title: " + rs.getString(columnLabel: "Job_Title"));
            }
        } catch (SQLException e) {
            System.err.println("Error accessing records: " + e.getMessage());
        }
    }
}
```

Manager Class

backend > src > models > Manager.java > Manager

```java
package models;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class Manager extends Employee {
    private int managedDepartmentID;

    public Manager(int userID, String username) {
        super(userID, username);
        this.role = "Manager";
        loadManagerSpecificData();
    }

    private void loadManagerSpecificData() {
        String query = "SELECT Department_ID FROM manager WHERE Manager_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.employeeID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                this.managedDepartmentID = rs.getInt(columnLabel: "Department_ID");
            }
        } catch (SQLException e) {
            System.err.println("Error loading manager data: " + e.getMessage());
        }
    }

    public void addRatings(int targetEmployeeID, double finalScore, String reviewPeriod) {
        String query = "INSERT INTO appraisal (Employee_ID, Final_Score, Review_Period) VALUES (?, ?, ?)";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, targetEmployeeID);
            pstmt.setDouble(parameterIndex: 2, finalScore);
            pstmt.setString(parameterIndex: 3, reviewPeriod);
            pstmt.executeUpdate();
            System.out.println("Rating added successfully for Employee ID: " + targetEmployeeID);
        } catch (SQLException e) {
            System.err.println("Error adding ratings: " + e.getMessage());
        }
    }

    public void addFeedback(int appraisalID, String comments, int ratingValue) {
        String query = "INSERT INTO feedback (Appraisal_ID, Manager_ID, Comments, Rating_Value) VALUES (?, ?, ?, ?)";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, appraisalID);
            pstmt.setInt(parameterIndex: 2, this.employeeID);
            pstmt.setString(parameterIndex: 3, comments);
            pstmt.setInt(parameterIndex: 4, ratingValue);
            pstmt.executeUpdate();
            System.out.println(x: "Feedback submitted successfully.");
        } catch (SQLException e) {
            System.err.println("Error adding feedback: " + e.getMessage());
        }
    }

    public void recordAchievement(int targetEmployeeID, String description, String date) {
        String query = "INSERT INTO achievement (Employee_ID, Achievement_Date, Description) VALUES (?, ?, ?)";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, targetEmployeeID);
            pstmt.setString(parameterIndex: 2, date);
            pstmt.setString(parameterIndex: 3, description);
            pstmt.executeUpdate();
            System.out.println(x: "Achievement recorded.");
        } catch (SQLException e) {
            System.err.println("Error recording achievement: " + e.getMessage());
        }
    }

    public void reviewAttendance() {
        String query = "SELECT e.Name, a.Date, a.Status FROM attendance a " +
                       "JOIN employee e ON a.Employee_ID = e.Employee_ID " +
                       "WHERE e.Department_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.managedDepartmentID);
            ResultSet rs = pstmt.executeQuery();

            System.out.println(x: "--- Department Attendance Report ---");
            while (rs.next()) {
                System.out.println("Employee: " + rs.getString(columnLabel: "Name") +
                                   " | Date: " + rs.getDate(columnLabel: "Date") +
                                   " | Status: " + rs.getString(columnLabel: "Status"));
            }
        } catch (SQLException e) {
            System.err.println("Error reviewing attendance: " + e.getMessage());
        }
    }

    public int getManagedDepartmentID() { return managedDepartmentID; }
}
```

Report Class

```
backend > src > models > Report.java > Report
package models;

import config.Database;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;

public class Report {
    protected int reportID;
    protected int employeeID;
    protected int generatedBy;
    protected Timestamp timestamp;
    protected String dataSummary;

    protected double averageRating;
    protected int totalAchievements;
    protected double attendancePercentage;

    public Report() {}

    public Report(int reportID) {
        this.reportID = reportID;
        loadReportData();
    }

    private void loadReportData() {
        String query = "SELECT * FROM report WHERE Report_ID = ?";
        try {
            Connection conn = Database.getInstance().getConnection();
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(parameterIndex: 1, this.reportID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                this.employeeID = rs.getInt(columnLabel: "Employee_ID");
                this.generatedBy = rs.getInt(columnLabel: "Generated_By");
                this.timestamp = rs.getTimestamp(columnLabel: "Time_Stamp");
                this.averageRating = rs.getDouble(columnLabel: "Average_Rating");
                this.totalAchievements = rs.getInt(columnLabel: "Total_Achievements");
                this.attendancePercentage = rs.getDouble(columnLabel: "Attendance_Percentage");

                this.dataSummary = "Base Report for Employee #" + employeeID +
                                   " | Rating: " + averageRating +
                                   " | Achievements: " + totalAchievements;
            }
        } catch (SQLException e) {
            System.err.println("Error loading report: " + e.getMessage());
        }
    }

    public String getDataSummary() {
        return this.dataSummary;
    }

    public void exportPDF() {
        System.out.println(x: "Exporting PDF...");
        System.out.println("Content: " + getDataSummary());
        System.out.println("Timestamp: " + this.timestamp);
    }

    public double getWorkforceProductivity() {
        return (averageRating * 20) + (attendancePercentage * 0.5);
    }

    public int getReportID() { return reportID; }
    public Timestamp getTimestamp() { return timestamp; }
}
```

According to our Sequence Diagram, the processes that we have implemented are –

- Initialization of Appraisal —> Recording of Feedback —> Eligibility Checking
- Promotion Approval —> Approval Notification to Employee

Discussion

While our project successfully implements several core features, the integration between our Java backend and PHP frontend remains a key area for future improvement. We originally planned a Tomcat-based architecture, but shifted to PHP to ensure a functional interface within our timeframe. As a result, certain Java methods exist as standalone logic within the backend but are not yet fully integrated into the web-side operations.